```
##
=========================================================================
===
## LAB EXPERIMENT: DEMONSTRATION OF CLUSTERING TECHNIQUES
## K-MEANS CLUSTERING AND HIERARCHICAL CLUSTERING (in R)
##
=========================================================================
===

## ------------------------------------------------------------------------
## AIM
## ------------------------------------------------------------------------
# To study and implement K-Means clustering and Hierarchical clustering
# techniques in R, and to compare the results obtained from both methods
# using the built-in "iris" dataset.

## ------------------------------------------------------------------------
## SOFTWARE / PACKAGES REQUIRED
## ------------------------------------------------------------------------
# Base R already contains kmeans() and hclust(). We use a couple of extra
# packages purely for nicer plots.

required_packages <- c("factoextra", "cluster", "dendextend")
new_packages <- required_packages[!(required_packages %in%
installed.packages()[,"Package"])]
if (length(new_packages) > 0) install.packages(new_packages)

library(factoextra)   # fviz_cluster, fviz_dend, fviz_nbclust
library(cluster)      # silhouette
```

```
library(dendextend)    # colour dendrogram branches

## ------------------------------------------------------------------------
## STEP 1: LOAD AND EXPLORE THE DATASET
## ------------------------------------------------------------------------
data(iris)
str(iris)
head(iris)
summary(iris)

# We use only the 4 numeric measurement columns for clustering.
# The 5th column (Species) is the TRUE label — kept aside for validation
# only, never used while clustering.
iris_data  <- iris[, 1:4]
true_label <- iris$Species

## ------------------------------------------------------------------------
## STEP 2: DATA PRE-PROCESSING (SCALING)
## ------------------------------------------------------------------------
# Clustering is distance-based, so features must be on comparable scales.
iris_scaled <- scale(iris_data)
head(iris_scaled)

##
=========================================================================
===
## PART A: K-MEANS CLUSTERING
```

```
##
================================================================================
===


## ------------------------------------------------------------------------
## STEP A1: DECIDE OPTIMAL NUMBER OF CLUSTERS (k)
## ------------------------------------------------------------------------
# (i) Elbow Method — plot Total Within-Cluster Sum of Squares (WSS) vs k
fviz_nbclust(iris_scaled, kmeans, method = "wss") +
  labs(title = "Elbow Method for Optimal k")

# (ii) Silhouette Method — higher average silhouette width = better k
fviz_nbclust(iris_scaled, kmeans, method = "silhouette") +
  labs(title = "Silhouette Method for Optimal k")

# Both methods typically suggest k = 2 or k = 3 for iris.
# Since we know there are 3 species, we proceed with k = 3.

## ------------------------------------------------------------------------
## STEP A2: APPLY K-MEANS
## ------------------------------------------------------------------------
set.seed(123)   # for reproducibility (initial centroids are random)
kmeans_result <- kmeans(iris_scaled, centers = 3, nstart = 25)

print(kmeans_result)

# Key components:
kmeans_result$centers   # final cluster centroids
kmeans_result$cluster   # cluster assigned to every observation
```

```
kmeans_result$size     # number of points per cluster
kmeans_result$withinss  # within-cluster SS for each cluster
kmeans_result$tot.withinss  # total within-cluster SS (lower = tighter clusters)

## ------------------------------------------------------------------------
## STEP A3: VISUALISE K-MEANS CLUSTERS
## ------------------------------------------------------------------------
fviz_cluster(kmeans_result, data = iris_scaled,
             geom = "point", ellipse.type = "convex",
             palette = "jco", ggtheme = theme_minimal(),
             main = "K-Means Clustering on Iris Data (k = 3)")

## ------------------------------------------------------------------------
## STEP A4: VALIDATE AGAINST TRUE SPECIES LABELS
## ------------------------------------------------------------------------
kmeans_table <- table(Predicted_Cluster = kmeans_result$cluster,
                      Actual_Species    = true_label)
print(kmeans_table)

kmeans_accuracy <- sum(apply(kmeans_table, 2, max)) / nrow(iris)
cat("Approx. K-Means clustering accuracy vs true species:",
    round(kmeans_accuracy * 100, 2), "%\n")
```

```
##
=====================================================================
===
## PART B: HIERARCHICAL CLUSTERING
##
=====================================================================
===


## ---------------------------------------------------------------------
## STEP B1: COMPUTE THE DISTANCE MATRIX
## ---------------------------------------------------------------------
dist_matrix <- dist(iris_scaled, method = "euclidean")


## ---------------------------------------------------------------------
## STEP B2: APPLY AGGLOMERATIVE HIERARCHICAL CLUSTERING
## ---------------------------------------------------------------------
# Try Ward's method (minimises total within-cluster variance)
hc_ward <- hclust(dist_matrix, method = "ward.D2")


# You can also try other linkage methods for comparison:
hc_single   <- hclust(dist_matrix, method = "single")
hc_complete <- hclust(dist_matrix, method = "complete")
hc_average  <- hclust(dist_matrix, method = "average")


## ---------------------------------------------------------------------
## STEP B3: PLOT THE DENDROGRAM
## ---------------------------------------------------------------------
plot(hc_ward,
    main = "Dendrogram - Hierarchical Clustering (Ward's Method)",
```

```
    xlab = "Observations", ylab = "Height", cex = 0.6)
rect.hclust(hc_ward, k = 3, border = c("red", "blue", "green")) # highlight 3 clusters

# Nicer coloured version using factoextra
fviz_dend(hc_ward, k = 3,
          cex = 0.5,
          k_colors = c("#E41A1C", "#377EB8", "#4DAF4A"),
          color_labels_by_k = TRUE,
          rect = TRUE,
          main = "Hierarchical Clustering Dendrogram (k = 3)")

## ---------------------------------------------------------------------
## STEP B4: CUT THE DENDROGRAM INTO k CLUSTERS
## ---------------------------------------------------------------------
hc_clusters <- cutree(hc_ward, k = 3)
table(hc_clusters)

## ---------------------------------------------------------------------
## STEP B5: VISUALISE HIERARCHICAL CLUSTERS (SCATTER FORM)
## ---------------------------------------------------------------------
fviz_cluster(list(data = iris_scaled, cluster = hc_clusters),
             geom = "point", ellipse.type = "convex",
             palette = "npg", ggtheme = theme_minimal(),
             main = "Hierarchical Clustering on Iris Data (k = 3)")

## ---------------------------------------------------------------------
## STEP B6: VALIDATE AGAINST TRUE SPECIES LABELS
## ---------------------------------------------------------------------
```

```
hc_table <- table(Predicted_Cluster = hc_clusters, Actual_Species = true_label)
print(hc_table)


hc_accuracy <- sum(apply(hc_table, 2, max)) / nrow(iris)
cat("Approx. Hierarchical clustering accuracy vs true species:",
   round(hc_accuracy * 100, 2), "%\n")
 ========================================================================
## PART C: COMPARISON OF THE TWO TECHNIQUES
##
========================================================================
===
comparison_table <- data.frame(
  Feature = c("Approach", "Need k in advance?", "Output",
           "Handles large datasets", "Sensitive to outliers/noise",
           "Reproducibility", "Iris accuracy (approx.)"),
  KMeans = c("Partitioning (centroid-based)", "Yes",
           "Flat set of k clusters", "Yes (fast, scalable)",
           "High", "Depends on random seed / initial centroids",
           paste0(round(kmeans_accuracy * 100, 2), "%")),
  Hierarchical = c("Agglomerative (bottom-up, tree-based)", "No (decide via
dendrogram)",
                "Dendrogram + flat clusters after cutting",
                "No (slow for large n, O(n^2) memory)",
                "Moderate (depends on linkage)",
                "Deterministic (same result every run)",
                paste0(round(hc_accuracy * 100, 2), "%"))
)
print(comparison_table)
```